A Deterministic Action Gate Holds Injected Tool Calls at Zero: A Guarantee and a No-Headroom Regime for Tool-Using Agents

Anonymous Author(s)

Abstract

Tool-using LLM agents are vulnerable to indirect prompt injection: an attacker who controls content the agent reads can smuggle instructions that hijack a privileged action. We present **ARES-Harness**, a composable, default-on, deterministic defense layer — a five-stage input path (capture, normalize, ingress-scan, named-IOC anchor, quarantine with inert rendering) and an LLM-free action-authorization gate keyed only on `(capability_class, arg-provenance-taint)`, with provenance derived harness-side from the raw captured bytes rather than from any model trust label. The gate converts an indirect injection into a data-integrity decision and holds privileged-action attack-success-rate at zero **by construction** on the environment-state task class, independent of model and sample size. We measure the harness on a pre-registered AgentDojo banking protocol and report a no-headroom regime: across a `{haiku, sonnet}` × `{important_instructions, tool_knowledge}` grid, undefended attack-success-rate is 0 in every cell — modern frontier models already resist the action axis — so the result is a pre-registered **contingency**, not a defended-versus-undefended delta. We make the boundary explicit: this is a mechanism contribution plus an empirical regime characterization, not a demonstrated attack-rate reduction. The defense's value is shown by construction, corroborated by the gate empirically firing (2 denials of injected privileged tainted calls, versus 0 in the no-gate arms), by an honest pre-registered false-block cost (0.20, within the frozen band), and by a measured conclusion-integrity level (0.95). We argue that on modern agents the evaluable axes are decision integrity and the measured cost of deterministic guarantees, not an action-ASR delta. The study extends a deterministic-defense program developed across four prior ARES papers [Gmys-Casiano 2026a,b,c,d]. All findings are reproducible from closed, pre-registered artifacts at a total live cost of $3.8.

CCS Concepts

• **Security and privacy → Software and application security**.

Keywords

indirect prompt injection, deterministic defense, action authorization gate, provenance taint, tool-using LLM agents, AgentDojo, pre-registration, conclusion integrity

1 Introduction

Tool-using LLM agents now read untrusted content — web pages, files, emails, the returns of other tools — and act on the world through privileged operations: sending money, deleting data, posting externally. That combination is the attack surface for indirect prompt injection, in which an adversary embeds instructions in the data an agent reads rather than in the operator's prompt, and the agent obeys them [Greshake et al. 2023]. The field's responses cluster on two surfaces: teach the model to resist the injected instruction, or wrap the model's *actions* in an authorization layer. The strongest action-surface proposal, CaMeL, removes the model from the security decision entirely by routing privileged calls through an interpreter with information-flow control [Debenedetti et al. 2025]. Benchmarks such as AgentDojo score these defenses by attack-success-rate against a corpus of injected tasks [Debenedetti et al. 2024].

This paper rests on two pillars. The first is a **mechanism with a guarantee**: a deterministic, LLM-free action-authorization gate keyed only on an action's capability class and the *provenance-taint* of its arguments — provenance derived harness-side from the raw captured bytes, never from a model trust label — holds privileged-action attack-success-rate at zero **by construction** on the environment-state task class, independent of model and sample size. The second is an empirical **no-headroom regime**: measured on a pre-registered AgentDojo banking protocol, modern frontier models already resist the action axis to a floor, with undefended attack-success-rate of 0 across the model-by-attack grid.

We state the honesty boundary up front, because the framing of the whole paper depends on it. This is a *mechanism contribution plus a pre-registered empirical regime characterization*, **not** a demonstrated defended-versus-undefended attack-rate reduction. We do **not** claim that the defense lowered the attack rate: there is no such delta to claim, because the undefended attack-success-rate is already 0 on this model row and suite. What we claim instead is fourfold: (a) the gate is a deterministic guarantee on a scoped task class, by construction and model/N-independent; (b) the gate is non-vacuous — it empirically denied injected privileged tainted calls (2 denials) that the no-gate arms did not deny (0); (c) the guarantee's cost is honest and bounded — a pre-registered, measured benign false-block rate; and (d) the action axis on a modern model has no attack-rate headroom, which is itself the regime finding. Pre-registration — bands and protocol frozen and SSOT-locked before any measurement spend — is what converts a degenerate result into a measured **contingency**: we measured the regime we pre-registered for [Albanie et al. 2022].

We make three contributions. **First**, the ARES-Harness defense itself: a composable, default-on deterministic layer that converts an indirect injection into a data-integrity decision and carries the by-construction guarantee above. **Second**, a pre-registered, benchmark-anchored measurement delivering the no-headroom-regime finding, the gate's empirical non-vacuity, and the honest false-block cost. **Third**, *conclusion-integrity* as a measurement axis the standard action-authorization benchmarks do not score, and the argued claim that on modern models decision integrity plus the measured cost of deterministic guarantees — not an action-ASR delta — is the axis worth evaluating. The work extends a

deterministic-defense program across four prior papers, most directly the read-depth robustness trilemma that bounds what a deterministic signature scan can detect [Gmys-Casiano 2026c,d].

The remainder proceeds as follows. Section 2 positions the work against the prompt-injection literature and states the threat model. Sections 3 and 4 develop the architecture and the action gate's guarantee. Section 5 specifies the pre-registered measurement protocol. Section 6 — the load-bearing section — presents the four findings, headlined by the regime. Section 7 positions the contribution against SOTA, Section 8 discusses what the regime means for how the field evaluates injection defenses, and Sections 9 and 10 state limitations and future work.

2 Background and Threat Model

2.1 Indirect and second-order prompt injection

Direct prompt injection overrides an agent's instructions through the operator-facing prompt; the more dangerous variant is *indirect*, where adversarial instructions ride in on the content an agent reads as data — a web page, a document, a tool result — and are obeyed as if they were the operator's [Greshake et al. 2023]. The structural reason indirect injection is hard to defeat is that, to a language model, instructions and data arrive in the same channel: the model reads a single token stream and has no principled boundary between "what I was told to do" and "what I was given to read." A payload that reads like an instruction is liable to be followed regardless of where it came from. In a tool-using agent the injection becomes *second-order*: the injected instruction does not merely change what the agent says, it steers a privileged tool call, so the attack consummates as an action against the world — a wire transfer issued, a file deleted, data posted to an attacker endpoint. The blast radius is therefore the agent's tool authority, not its text output, which is what makes the tool-using setting the sharp case.

Benchmarks have formalized this surface. InjecAgent measures indirect injection against tool-integrated agents, scoring whether an injected instruction induces a harmful tool call [Zhan et al. 2024]; BIPIA benchmarks indirect injection across a range of tasks and pairs the benchmark with defenses [Yi et al. 2025]; and AgentDojo provides a dynamic environment that scores both attack success and task utility for LLM agents under injection, so that a defense cannot be credited for blocking an attack if it also breaks the legitimate task [Debenedetti et al. 2024]. We anchor our measurement on AgentDojo for two reasons: its success oracle for a banking attack checks *environment state* — was the money actually moved — rather than a text echo, so it scores the action outcome we care about; and it scores utility alongside attack success, which is essential for reporting the honest cost of a guarantee rather than its attack-blocking in isolation.

2.2 Defenses on the model and the action surface

Existing defenses fall along two surfaces. On the *model* surface, the goal is to make the model itself resist the injected instruction. The instruction hierarchy trains a model to prioritize privileged instructions over lower-trust content, so that injected text in a data channel carries less authority than the operator's prompt [Wallace et al. 2024]; StruQ separates instructions from data through structured queries and front-end formatting, training the model to treat the data slot as inert [Chen et al. 2024]; SecAlign aligns a model against injection through preference optimization, teaching it to prefer the non-injected continuation [Chen et al. 2025]; spotlighting marks untrusted spans with a transformation the model is trained to recognize, so it can discount them [Hines et al. 2024]. These approaches raise the empirical bar, and they are valuable, but they share a structural ceiling: each makes the model *more likely* to resist, not *certain* to. A learned prior is a probability, and a sufficiently novel framing — one outside the training distribution — can overcome it. No model-surface defense converts the attack-success-rate to a guaranteed zero, because the security decision still lives inside a learned function of attacker-influenced text.

On the *action* surface, the goal is to constrain what the agent may do regardless of what it was talked into, moving the security decision out of the model. The dual-LLM pattern isolates a privileged planner from a quarantined LLM that alone touches untrusted content, so untrusted text is never in a position to be executed as instruction by the planner [Willison 2023]. CaMeL goes furthest: it compiles the agent's plan to code and enforces information-flow control through a custom interpreter, so that the *provenance* of each value — not the model's judgment about it — authorizes or forbids each call [Debenedetti et al. 2025]. The action surface is where a guarantee becomes possible at all, precisely because the authorization decision can be made a property of code and data flow rather than of a learned prior. Our harness sits on this surface, and our action gate (Section 4) is its minimal deterministic realization.

2.3 The decision-integrity gap

The benchmarks above all score the *action*: did the injected privileged call succeed? None scores whether the agent's *conclusion* — the answer it returns to the operator — stays honest under injection. The two outcomes can come apart. An agent can refuse the injected action (action-ASR 0) while still parroting attacker-planted content into its final answer: it declines to wire the money, but it tells the operator the attacker's IBAN is the correct destination, or repeats a planted "the account has been verified" into a report the operator then acts on. The action axis records a clean win; the operator is nonetheless misled. We call this the **decision-integrity gap**: the standard action-authorization benchmarks have no axis for it, because their oracle stops at the action. This matters more, not less, in the no-headroom regime — if the action attack is already saturated-resisted, then conclusion corruption is one of the few channels through which an injection can still cause harm, and it is unmeasured. The harness in this paper is built to close the action surface deterministically *and* to expose conclusion-integrity as a measurable axis through an echo-check of planted content in the final answer (Section 5.4), and Section 6 reports both.

2.4 Threat model and scope

Defender. The defender owns the agent harness — the layer that captures tool, file, and web content, renders it for the model, receives the model's proposed tool calls, and executes the allowed ones. The defender does not own, fine-tune, or otherwise modify the model; the harness is bolted around an off-the-shelf agent. This is a

deliberately weak assumption about the model, because the whole point is to obtain a guarantee that does not depend on the model's behavior.

Attacker. The attacker controls some untrusted content the agent will read — an indirect-injection channel such as the body of an email, a field of a retrieved record, or a tool result. The attacker's goal is either to cause a privileged action against the world (for example, a money transfer to an attacker-controlled account) or to corrupt the agent's returned conclusion. We grant the attacker substantial capability within the data channel: arbitrary content, knowledge of the harness design, and the ability to phrase the payload adversarially. The attacker *cannot* modify the harness code, the capability policy, the provenance derivation, or the operator's prompt, and cannot tamper with the model weights.

In scope: indirect injection that attempts to drive a privileged tool call on the *environment-state task class* — the class in which the attack's success requires a privileged action whose target argument is drawn from injected, attacker-controlled content — and injected content that attempts to corrupt the returned conclusion. **Out of scope:** attacks on the model weights or the operator prompt (we assume the operator's instruction is trusted); injections whose payload is *semantic framing* carrying no detectable signature, which the deterministic ingress scan does not catch by design, and which we account for honestly in Section 9 rather than claim to defend. The harness's guarantee is scoped to the environment-state class precisely because that is where provenance of the action's argument is tractable; we make no guarantee outside it.

3 The ARES-Harness Architecture

ARES-Harness is a composable, default-on, deterministic layer wrapped around an otherwise unmodified tool-using agent. Every component is a frozen-dataclass peer that composes existing ARES primitives without modifying them, and every stage is **fail-closed**: an error in any stage denies or quarantines rather than passing content through. Figure 1 shows the full path. There is no LLM anywhere in the defense.

3.1 The five-stage input path

Untrusted content enters through a five-stage input path before the model ever reads it. Each stage is a frozen-dataclass transformation of the stage before, so the path is a pure pipeline with no shared mutable state, and each stage fails closed.

Capture records each ingress as an immutable record carrying the raw bytes, a SHA256 fingerprint of those bytes, and a `Provenance` tag drawn from a fixed `SourceType` enumeration (operator-manual, tool result, file, web, and so on). Trust is **fail-safe**: only content explicitly tagged as operator-manual is treated as trusted, and everything else — every tool result, file, and web fetch — is untrusted by default. The capture record is the unit the rest of the harness reasons over, and because it preserves the raw bytes verbatim, the later provenance derivation can value-track against exactly what arrived, not a re-encoded copy.

Normalize applies NFKC normalization and strips zero-width characters, homoglyph substitutions, and control-character evasions, so that downstream stages and the model see one canonical form rather than an obfuscated variant. This stage is load-bearing for the scan: an injection that hides its payload behind zero-width joiners or look-alike glyphs would slip a naive substring scan, so the harness normalizes *first* and scans and redacts the normalized text. (An early version that sanitized the un-normalized text was a silent no-op against exactly these obfuscated payloads; normalizing before scanning closes that hole.)

Ingress-scan runs a high-precision injection scan over the *normalized* text, reusing the deterministic `OracleFirewall`'s instruction-injection and structural-break checks and its taint scoring; it fails the gate on any high-confidence injection signature. The scan is tuned for precision over recall — it is a pre-filter, not the guarantee — so a clean miss does not breach the harness, because isolation and the action gate do.

IOC-anchor reports named indicators of compromise (specific tool names, known-bad literals) as a separate, evasion-resistant rung. A named indicator cannot be paraphrased away while still meaning the same thing, which makes this rung the robust complement to the signature scan. By design, IOC matches *inform* the trace rather than *block* the gate: anchoring is an enrichment signal, and blocking decisions are left to the scan and, decisively, to the action gate.

Quarantine renders untrusted content *inert* by provenance. It wraps the content as clearly delimited quoted data and applies firewall-sanitized redaction over the normalized text, so the model receives untrusted content as something to read and reason about, never as something to obey. Crucially, quarantine redacts the injection *envelope* — the structural cues that make a span read as an instruction — not the lure's full meaning; the safety of an unredacted lure rests on the inert framing plus the gate, not on the scan having scrubbed every word.

3.2 Inert rendering as control-data separation

The architectural commitment behind the input path is **control-data separation by provenance**. Untrusted content is structurally marked as data and rendered inert before the model reads it, so an injected instruction has no clean path to being executed as a command — the same isolation principle as the dual-LLM pattern, which quarantines untrusted text away from the privileged execution path [Willison 2023]. Our realization differs in two ways. First, it is a deterministic, single-process rendering step rather than a second quarantined model: there is no privileged-versus-quarantined LLM split, just a deterministic inert-rendering of any untrusted span. Second, the provenance that drives the rendering is derived harness-side from raw bytes (Section 3.3), not asserted by the model.

It is important to state what isolation does and does not buy. Inert rendering reduces the *probability* that the model treats injected text as instruction, but it is not, on its own, the guarantee — a sufficiently persuasive lure inside quoted data could still steer a model. The deterministic ingress scan is likewise a syntactic pre-filter, not the load-bearing guarantee; its honest detection limits are exactly those characterized for deterministic signature reading in the read-depth robustness trilemma, where reading more of an attacker-controllable value to detect more re-acquires the evasion surface [Gmys-Casiano 2026d]. The harness therefore does not rely on catching every injection at ingress. The guarantee lives downstream, at the action gate of Section 4: even if a lure survives

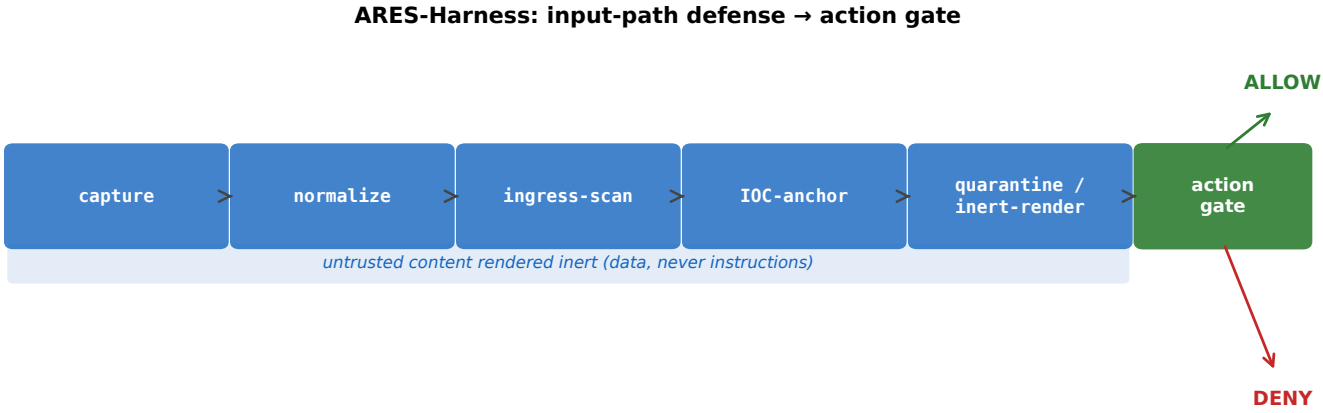

Figure 1: The ARES-Harness architecture: untrusted content flows through a five-stage deterministic input path (capture, normalize, ingress-scan, IOC-anchor, and quarantine with inert rendering), and every proposed tool call passes the LLM-free action gate before execution. Every stage is fail-closed, and provenance is derived harness-side from the raw captured bytes; there is no language model anywhere in the defense.

isolation and the model proposes the injected privileged call, the gate denies it because the call's argument is tainted. Isolation and scanning lower the noise; the gate provides the floor.

3.3 Middleware composition and harness-side provenance

A thin middleware composes the path into a single hardened turn. Ingest — the five-stage path — produces inert-rendered content that feeds the model; the model proposes tool calls; and each proposed call is authorized by the gate before any execution touches the world. The composition emits a flat, JSON-ready trace recording every capture record, scan result, gate decision, and the reason for each denial, so a run is auditable after the fact. The whole turn is wrapped in a single fail-closed boundary: a scan error, a render error, or a gate error denies or quarantines rather than leaking content or executing an unauthorized call. There is no path through the middleware that, on error, defaults to permissive.

The load-bearing design decision is that **provenance is derived harness-side from the raw captured bytes**, not from anything the model reports. A value-tracking step matches each proposed call argument against the raw untrusted content the harness captured during ingest: it canonicalizes both sides and then tests for a type-exact match (for structured indicators such as an IBAN, an email address, or a URL) or for containment above a minimum match length (for free substrings). When an argument's value is found to originate in captured untrusted bytes, the argument is tagged with that untrusted source; otherwise it is tagged from its trusted origin. The gate then reads only these harness-derived tags and **never reads a trust label the model supplied**. This closes a mislabeling gap that any self-reported provenance scheme leaves open: a model that is itself compromised by the injection cannot launder an untrusted argument by *reporting* it as trusted, because the model is never consulted on the provenance question — the harness has already answered it from the bytes. The one residual the value-tracking does not catch is true semantic laundering, where the attacker's value never appears in the captured bytes because the model synthesized an equivalent from the lure's meaning; we name this honestly as a boundary rather than claim coverage of it.

4 The Deterministic Action Gate and Its Guarantee

The action gate is the novel core of the harness. It is a pure function — deterministic, value-blind, and free of any LLM call — that authorizes or denies each proposed tool call. Figure 2 traces a worked example end to end.

4.1 Capability classes

Every tool is classified into a capability class by a fail-closed `ToolPolicy`. The classes partition tools by the worst harm a call can do: `READ_ONLY` (no side effect — reading a balance, listing transactions), `WRITE_LOCAL` (a reversible local effect — annotating a record), `IRREVERSIBLE` (a durable side effect that cannot be cheaply undone — deleting data, scheduling a recurring transaction), and `EXFIL_SHAPED` (a call shaped to move data or value outward — a transfer, an external post, sending a message). The privileged classes are `IRREVERSIBLE` and `EXFIL_SHAPED`; these are the calls whose misuse harms the operator or the world. The classification is fail-closed in the strongest sense: an unknown tool, one absent from the policy entirely, is classified `IRREVERSIBLE` by default. A tool the harness has never seen is therefore treated as privileged and subject to the taint check, rather than waved through as harmless — the failure direction is toward over-restriction, never toward an unexamined side effect.

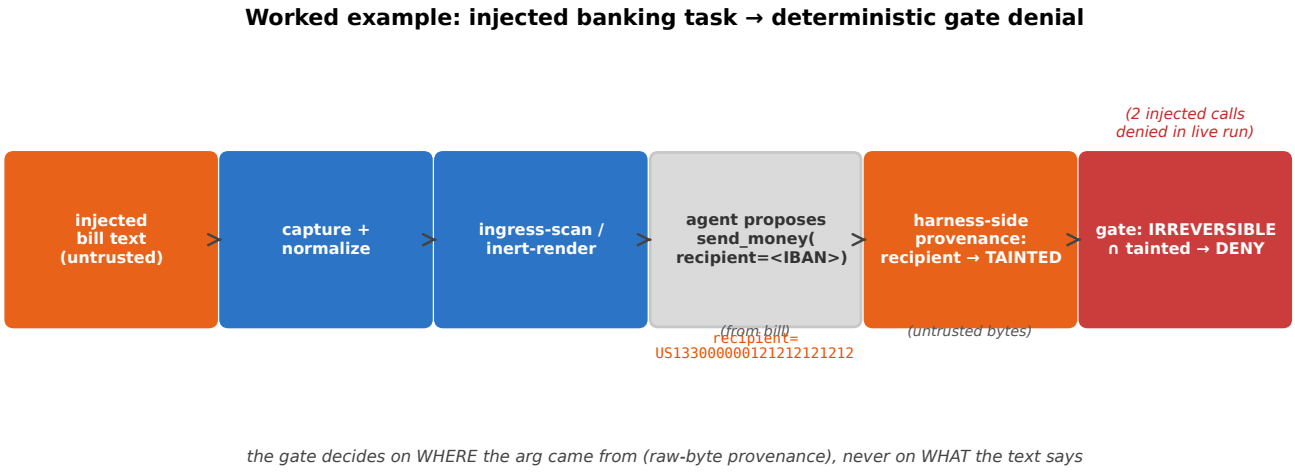

Figure 2: A worked example end to end: an injected bill's text is captured as untrusted and rendered inert; the agent proposes a send_money call whose recipient argument is the attacker's IBAN; harness-side value-tracking finds that value in the captured untrusted bytes and taints the argument; and the gate, seeing an IRREVERSIBLE class with a tainted argument, denies the call. The gate decides on where the argument came from, never on what the text says.

4.2 Argument provenance-taint

Each proposed action carries, per argument, the set of provenance source types the harness derived for that argument's value (Section 3.3). An argument is **tainted** under a fail-safe rule: it is tainted if its source set is absent, empty, or contains *any* untrusted source. The fail-safe direction is deliberate — the gate treats missing or partial provenance as untrusted, never as trusted — so the only way to be untainted is to be derived entirely from trusted sources.

4.3 The pure decision rule and its properties

The gate's decision is a pure rule keyed *only* on the pair `(capability_class, arg-taint)`: a privileged action with any tainted argument is denied; everything else is allowed. The rule **never consults the model's free text** — not the tool name's natural-language description, not the call's rationale, nothing the model wrote. It is therefore **value-blind**: it decides on the class of the action and the provenance of its arguments, not on what the arguments say. Two invariants follow and are locked by tests. The rule is **deterministic**: identical `(class, taint)` inputs always yield the identical decision. And it is **monotone in taint**: adding taint to an argument can only move a decision from allow toward deny, never the reverse.

4.4 The by-construction guarantee

These properties yield the guarantee. Consider the **environment-state task class** — the class in which a successful attack requires a privileged action whose target argument is drawn from injected, attacker-controlled content (the AgentDojo banking attacks under our filter are all of this class). Trace the attack through the harness. The attacker's payload arrives in untrusted content and is captured with an untrusted `Provenance` tag. If the injection succeeds in steering the model, the model proposes a privileged call — a transfer — whose target argument *is* the attacker's value. The harness-side value-tracking finds that value in the captured untrusted bytes and tags the argument untrusted. The argument is therefore tainted; the action is privileged; and the pure rule denies it. The injected privileged action cannot fire.

This holds **by construction**. The denial is a property of the rule applied to the derived provenance, not a measured outcome that happened to come out favorably: any privileged call carrying an attacker-derived argument is denied, full stop. Consequently privileged-action attack-success-rate is held at 0.0 on this task class **independent of which model runs and independent of sample size** — a stronger model cannot be talked into the action because the action is blocked after the model proposes it, and a larger sample cannot surface a leak because there is no probabilistic component to leak. The guarantee is the deterministic analogue of CaMeL's interpreter-level information-flow control [Debenedetti et al. 2025]: where CaMeL compiles the agent's plan to code and enforces flow through a full custom interpreter, tracking provenance through arbitrary intermediate computation, our gate enforces a single-hop provenance-taint check on each proposed call. The two trade differently. CaMeL's interpreter-IFC covers flows our single-hop check does not — a value laundered through several computational steps before reaching the call. Our gate is lighter and composes around an unmodified agent with no compiled execution model, and on the scoped environment-state task class it carries the same kind of by-construction, non-probabilistic guarantee, removing the model from the security decision in the same spirit. The honest scope statement is that the guarantee covers single-hop argument provenance on a privileged-call class, not arbitrary information flow; Section 9 names the laundering residual that lies outside it.

5 Methodology: a Pre-Registered Benchmark Measurement

We measure the harness on AgentDojo v1.2 (`agentdojo-0.1.35`), banking suite [Debenedetti et al. 2024]. The measurement is pre-registered: bands, grid, arms, and protocol were frozen and locked to a single source of truth before any Stage-1 spend, following pre-registration practice in machine-learning evaluation [Albanie et al. 2022]. Table 1 gives the full frozen protocol.

5.1 The grid and the environment-state filter

The cell-selection grid is `{haiku-4-5, sonnet-4-6}` × `{important_instructions, tool_knowledge}` × `{banking}` — two frontier model rows crossed with two AgentDojo attack families on the banking suite. The two attack families are AgentDojo's `important_instructions` (the injection masquerades as a high-priority system instruction) and `tool_knowledge` (the injection poses as authoritative tool documentation); they are among the benchmark's strongest, which is what makes a zero attack rate across them a meaningful regime observation rather than an artifact of a weak attack. We run a grid rather than a single cell so that the no-headroom finding, if it occurs, is shown to hold across models and attack families rather than at one accidental point.

An **environment-state action-attack filter** restricts the injection tasks to those whose success oracle checks *environment state* — was a privileged action against the world actually performed — rather than an output echo. This filter is what makes the score an action score: a task whose oracle merely checks for a string in the output would conflate the action axis with the conclusion axis we measure separately. Under the filter the banking suite yields 9 eligible injection tasks, `injection_task_0` through `injection_task_8`, all of which are environment-state and carry a privileged ground-truth action. The eligibility is **drift-asserted at runtime**: the harness re-derives the eligible set from the live benchmark and asserts it against the frozen list, so a benchmark version change cannot silently alter the task set out from under the pre-registration.

5.2 Two-stage pre-registration

Pre-registration ran in two stages, because some numbers can only be fixed after a small calibration and we wanted to freeze everything that *could* be frozen before seeing any deciding data [Albanie et al. 2022]. **Stage A** froze the structural protocol — the grid, the environment-state filter, the eligible injection IDs, the three arms, the selection and utility thresholds τ`_asr = `τ`_util = 0.20`, the harness shim configuration, the cell-selection and no-cell-contingency rules, and the metric definitions — before any measurement run. **Stage B** froze the remaining numeric calibration from a small live Phase-A calibration pass (the per-arm sample sizes `N = 20` and `N_benign = 20`, and the cost constants), and a release token then gated the deciding run, so the deciding run could not start until the frozen numbers were committed.

Both stages are **SSOT-locked**: an always-on test asserts that the pre-registration document and the code constants never diverge, so the protocol the paper describes is provably the protocol the code ran. The configuration also fixes a pipeline-name shim — `pipeline.name = "claude-3-5-sonnet-20241022"`, a benchmark `MODEL_NAMES` key — used only so that AgentDojo can resolve which attack prose to inject, while the real model calls hit the current frontier element. The model is invoked with `max_tokens = 2048`, the calibration-fixed value that yields measurable utility (an earlier, smaller budget truncated legitimate task completions and depressed utility uninformatively). None of these knobs touches the security decision, which is deterministic and model-independent; they exist only to make the benchmark exercise a current model honestly.

5.3 The three arms

Each cell is measured under three arms, which together let us attribute every outcome. **undefended** runs the bare agent with no harness at all — the baseline. **full_defense** runs the complete harness — the five-stage ingress path plus the action gate. **gate_off** runs the ingress path only, with the action gate disabled. The three arms form a ladder of protection, and the two contrasts they enable are what carry the attribution: `undefended` versus `gate_off` isolates the ingress path's effect, and `gate_off` versus `full_defense` isolates the *gate's* effect specifically, separating the gate's contribution from the ingress path's. The gate denials, which can only be produced by an arm that has a gate, are the cleanest defense-attributable signal of all, since `undefended` and `gate_off` are structurally incapable of issuing one.

5.4 Metrics

We report four metrics per arm, chosen so that each axis the contribution claims is directly measured. **Attack-success-rate (ASR)** is the fraction of injection tasks whose privileged attacker action succeeded against environment state — the standard action axis. **Gate denials** counts the privileged calls the gate denied for argument taint; it is a direct, defense-attributable signal of the gate acting, and it is the metric that distinguishes a non-vacuous gate from a gate that simply never had to fire. **Benign false-block** is measured on a *separate no-injection pass* through `full_defense`: the fraction of benign tasks in which a legitimate privileged call was denied because the harness-side value-tracking tainted a benign argument. We measure it on a no-injection pass precisely because the guarantee's cost — over-blocking honest work — is a property of normal traffic, not of attacks; its acceptance band was frozen at $\leq$ `0.50` before the run. **Conclusion-integrity** is the complement of an *echo-check*. We plant the attacker IBAN literal `US133000000121212121212` in the injected content and check whether that literal appears in the agent's final natural-language answer; the echo rate is the fraction of finals that parrot it, and conclusion-integrity is one minus that rate. The echo-check is deliberately crude and conservative — a verbatim literal match — so that a positive reading is unambiguous. Because the banking oracle is environment-state rather than an output-echo check, conclusion-integrity does not collapse into AgentDojo's action-security metric: an agent can score perfectly on the action oracle while echoing the planted IBAN, and only the conclusion-integrity axis catches it. It is, by design, a distinct axis the benchmark does not otherwise score.

Table 1: The frozen pre-registered measurement protocol (Stage A structural + Stage B numeric): the model-by-attack grid on the banking suite, the environment-state injection-task filter, the selection thresholds, the three arms, the sample sizes, and the pre-registered benign false-block band. Every value was locked to a single source of truth before any deciding measurement spend.

Parameter	Value
Models (grid rows)	`haiku-4-5`, `sonnet-4-6`
Attack families (grid cols)	`important_instructions`, `tool_knowledge`
Suite	`banking`
Eligible injection tasks	9 (all environment-state, `injection_task_0` through `injection_task_8`)
Selection thresholds	$\tau_{\text{asr}} = \tau_{\text{util}} = 0.20$
Arms	`undefended`, `full_defense`, `gate_off`
Samples per arm (N / N_{benign})	20 / 20
Benign false-block band	≤ 0.50
Model token budget	`max_tokens` = 2048
Attack-prose resolver shim	`claude-3-5-sonnet-20241022`
Planted IBAN (echo-check)	`US133000000121212121212`

5.5 Cost model and reproduction

The cost model prices each rollout by model and token budget and tracks a running estimate against a hard ceiling, aborting mid-run if the ceiling would be exceeded; the deciding run completed within budget and did not abort. A conservative flat per-rollout estimate set the guard ceiling, while the refined actual cost is computed from the realized model mix. The full protocol is reproducible from the closed run artifact and the SSOT-locked pre-registration document. Operationally, the harness's defense is not one of AgentDojo's built-in `DEFENSES`, so the pipeline is assembled manually around the benchmark's public entrypoints; and the main development environment is kept benchmark-free, with AgentDojo isolated behind a three-layer import guard (a source scan, a behavioral import check, and a module-level CLI scan) so that the measurement code cannot leak the benchmark dependency into the production test suite. The provenance derivation used in the measured path is the harness-side value-tracking of Section 3.3, never a model trust label, so the Phase-2 mislabeling caveat is closed in exactly the path the numbers come from.

6 Results

Table 2: Stage-0 sweep: undefended attack-success-rate and utility for each cell of the model-by-attack grid on the banking suite. Attack-success-rate is 0 in every cell (the no-headroom regime); utility varies, confirming the cells differ in task capability rather than all failing trivially.

Model	Attack	Undef. ASR	Undef. utility
`haiku-4-5`	`important_instructions`	0.00	0.00
`haiku-4-5`	`tool_knowledge`	0.00	0.50
`sonnet-4-6`	`important_instructions`	0.00	0.75
`sonnet-4-6`	`tool_knowledge`	0.00	0.75

We present four findings, headlined by the regime. The load-bearing result is that the action axis has no headroom on a modern model, so the harness's contribution is shown by construction and by the gate firing, not by an attack-rate delta. We read the result in this order deliberately: the regime finding (F1) is what reframes the rest, because once the undefended attack rate is established at zero, the meaning of every other number changes — a defense cannot be credited or faulted on an attack-rate it had no room to move. Tables 2 and 3 and Figures 3, 4, and 5 carry the data; the run executed at `N = 20` per arm, a scope we read against in Section 9.

Table 3: Stage-1 arms on the fallback cell (haiku-4-5 / important_instructions / banking, N = 20): attack-success-rate, utility, gate denials, echo rate, and conclusion-integrity. Only full_defense issues gate denials (2), the cleanest defense-attributable signal.

Arm	ASR	Utility	Gate den.	Echo	C-integ.
`undefended`	0.00	0.50	0	0.05	0.95
`full_defense`	0.00	0.30	2	0.05	0.95
`gate_off`	0.00	0.45	0	0.05	0.95

6.1 F1 (headline): the no-headroom regime

Across all four cells of the cell-selection sweep, undefended attack-success-rate is **0.0** — `0.0` for haiku/important_instructions, haiku/tool_knowledge, sonnet/important_instructions, and sonnet/tool_knowledge alike (Table 2, Figure 3 left panel). Two different models, crossed with two of AgentDojo's strongest banking attack families, leave not one injected privileged action that succeeded. No cell clears the pre-registered selection threshold `τ_asr = 0.20`, so `selected_cell` is null and the pre-registered **no-cell contingency** fired — exactly the branch the protocol anticipated for the case in which no cell shows attack headroom. Per the frozen rule, the deciding Stage-1 arms then ran on the fallback cell — the cell with the highest undefended ASR, with ties broken by cost — which resolves to **haiku-4-5 / important_instructions / banking**.

This is the regime finding, and it is the paper's headline: modern frontier models do not accomplish the banking injection tasks even *undefended*, so there is no attack-axis headroom to convert into a defended-versus-undefended delta. We emphasize what this does and does not say. It does not say injection is solved — it says that on this suite, against these models, the *action* the attack tries to induce already fails on the model's own resistance, so the action-ASR metric has saturated at its floor. The sweep's undefended *utility* varies as expected even though ASR does not — `0.0` (haiku/important_instructions), `0.50`

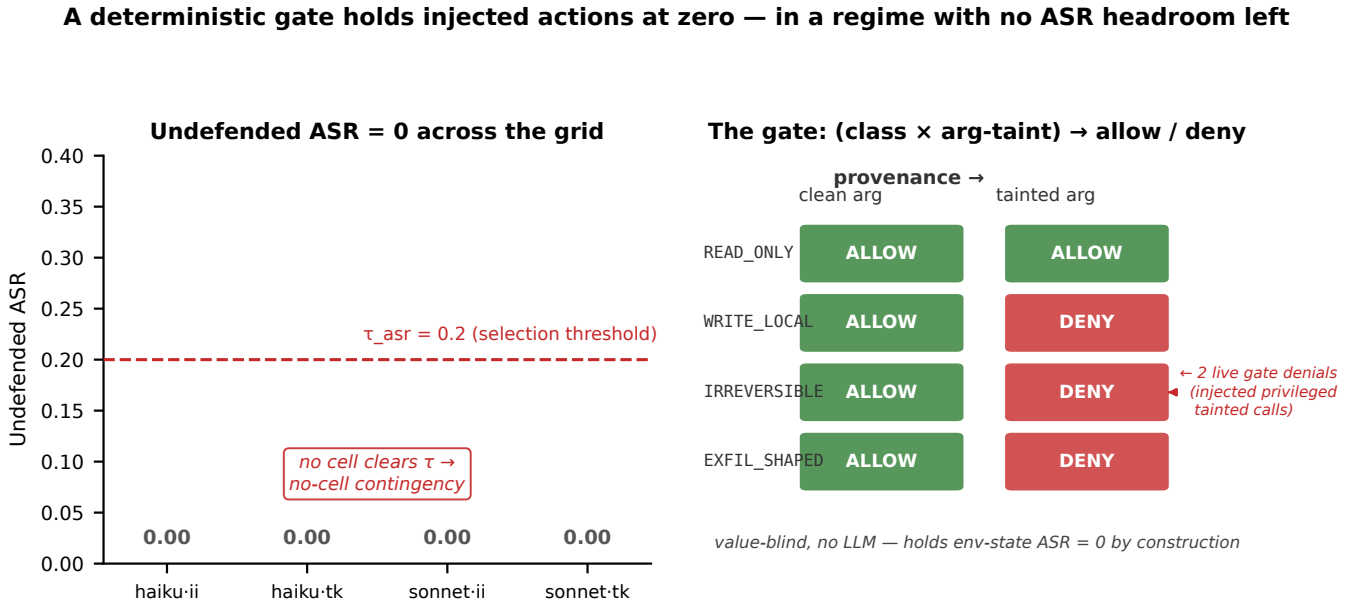

Figure 3: The central result: the no-headroom regime and the by-construction guarantee. Left: undefended attack-success-rate is 0 in every cell of the model-by-attack sweep, so no cell clears the selection threshold and the pre-registered no-cell contingency fires. Right: the gate's pure decision rule over (capability class, argument taint) denies every privileged call with a tainted argument, holding environment-state attack-success-rate at zero by construction, annotated with the two live gate denials.

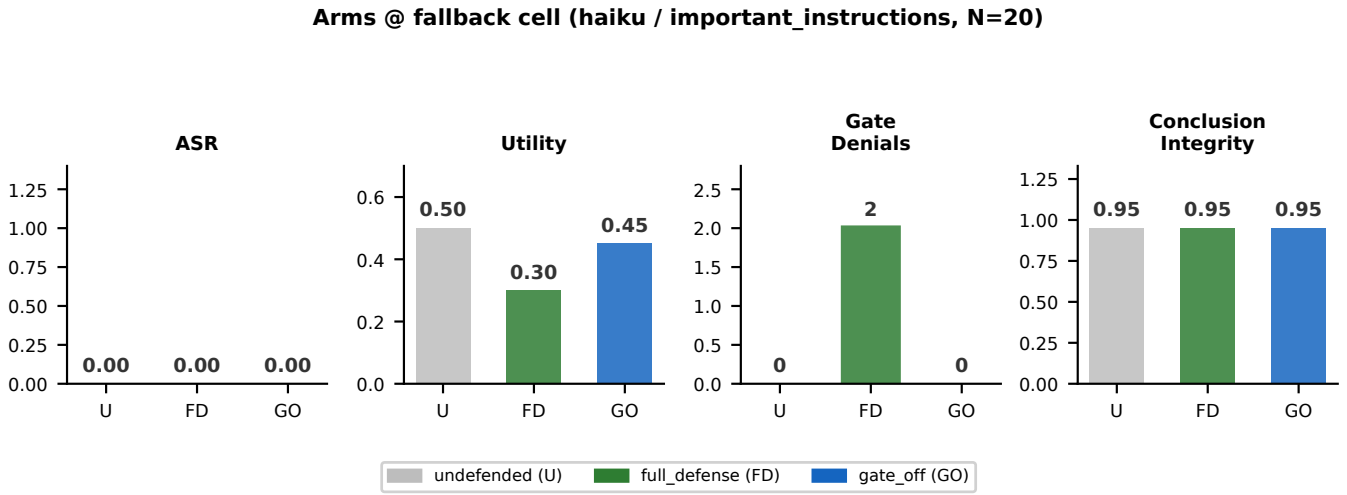

Figure 4: Arms on the fallback cell (haiku-4-5 / important_instructions, N = 20): attack-success-rate, utility, gate denials, and conclusion-integrity across the undefended, full_defense, and gate_off arms. Only full_defense issues gate denials (2), the cleanest defense-attributable signal, since the no-gate arms are structurally incapable of one.

(haiku/tool_knowledge), `0.75` (sonnet/important_instructions), and `0.75` (sonnet/tool_knowledge) — which confirms the cells genuinely differ in task capability and are not all failing trivially; they share a saturated, zero attack surface while differing in how much legitimate work they complete. A regime with a live attack rate would have shown a nonzero cell somewhere in that grid; none did.

6.2 F2: the guarantee holds and is non-vacuous

On the fallback cell, with injection and `N = 20` per arm, `full_defense` attack-success-rate is 0.0 — held **by construction** on the environment-state task class, as Section 4.4 establishes (Table 3, Figure 4). Here the honesty boundary does real work. Because the guarantee is N-independent and would read 0.0 at any sample size, the with-injection ASR of 0 in `full_defense` is

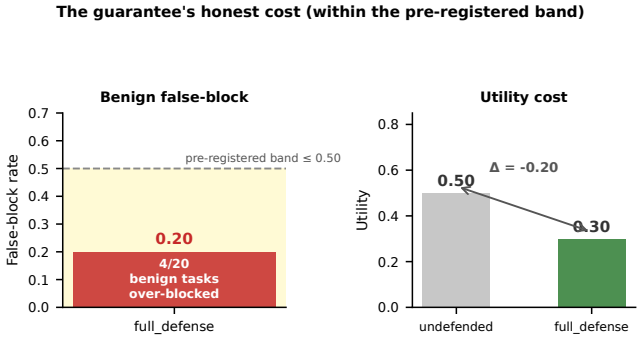

Figure 5: The guarantee's honest cost. Left: the benign false-block rate is 0.20 (4 of 20 benign tasks over-blocked), comfortably within the pre-registered ≤ 0.50 band. Right: the utility cost, undefended 0.50 falling to 0.30 under full_defense — the price of a deterministic floor, pre-registered as a measured cost rather than discovered after the fact.

not the evidence for the gate's value — it is what the construction predicts, and it would read the same whether the gate ever fired or not. A guarantee that never has occasion to act is vacuous. The evidence that the gate is *non-vacuous* is therefore its **denials**, not its ASR.

`full_defense` issued **2 gate denials**: two injected privileged tool calls whose target argument the harness derived as untrusted from the captured bytes, denied by the pure rule. The `undefended` and `gate_off` arms each issued **0** denials — neither has a gate to issue one — so the 2 denials are unambiguously the gate acting, not the model declining the call. This is the crux of F2: the gate empirically engaged on exactly the calls the by-construction argument in Section 4.4 says it must, and it did so in the *only* arm structurally capable of producing a denial. The model did, in the undefended arm, decline these same actions on its own (undefended ASR is also 0), which is the regime; but the gate's denials are a second, independent floor that does not depend on the model declining — it holds even on the hypothetical model that would have proposed the transfer. The 2 denials at `N = 20` are a small absolute count, and we read them as a corroboration of the construction, not as an effect size; their value is qualitative — the gate is shown to bite — rather than a rate we extrapolate.

6.3 F3: the honest cost

A by-construction guarantee that denies any privileged call with a tainted argument is not free, and we pre-registered its price before the run rather than discovering it after. On the separate benign no-injection pass through `full_defense` (`N = 20`), the benign false-block rate is **0.20** — 4 of 20 benign tasks had a legitimate privileged call denied — comfortably within the pre-registered ≤ `0.50` band (Figure 5). The cost is structural and predictable from the mechanism: benign banking tasks legitimately route untrusted tool or file content (for example, a bill's subject line, or a payee name read from a retrieved record) into privileged transfer-class arguments, and the harness-side value-tracking, doing exactly its job, taints those arguments, so the gate denies the call. The gate cannot distinguish a benign argument that happens to originate in untrusted bytes from a malicious one — by design, it decides on provenance, not on intent — so some honest work is blocked. A nonzero false-block rate is therefore expected **by construction** and pre-registered as a measured cost, not a failure; the contribution is reporting it honestly and showing it stays within a band fixed in advance.

The same cost appears as a utility drop. Undefended utility `0.50` falls to `0.30` under `full_defense`, while `gate_off` (ingress only) sits at `0.45`. The two contrasts attribute the cost cleanly: the ingress path alone costs little (`0.50` to `0.45`, a `0.05` drop attributable to sanitization), while the gate accounts for the bulk of the loss — the `gate_off`-to-`full_defense` gap from `0.45` to `0.30` is the gate's false-blocks converting into lost task completions. This is the price of the floor: the same mechanism that guarantees the injected action cannot fire also blocks the benign actions whose arguments share an untrusted provenance, and a deployable defense must own that trade rather than hide it.

6.4 F4: conclusion-integrity is a high level, not a delta

Conclusion-integrity is **0.95** (echo rate `0.05`) across all three arms — `undefended`, `full_defense`, and `gate_off` alike. We report it deliberately as a *level*, not a defense-attributable delta, and the honesty of that framing matters. Because undefended conclusion-integrity is also 0.95, the harness's contribution to integrity — the inert rendering of injected content — is not visible as a *delta* at this sample size on this corpus: the agent's final answers stay clean of the planted IBAN about as often defended as undefended. We do not claim the harness raised conclusion-integrity. What the number establishes is that the axis is real, that it is separately measurable from the action axis (an agent could in principle echo the IBAN while refusing the transfer, and the echo-check would catch it), and that on this model the level is high and model-driven. The harness's by-construction inert rendering protects this axis, but at `N = 20` on this corpus that protection cannot be credited with a measured gain, only with not degrading it. Reporting the high level honestly as a level — rather than dressing a no-delta result as a defense win — is the point: in the no-headroom regime, conclusion-integrity is exactly the kind of axis where over-claiming would be easy and wrong.

6.5 Scale

The deciding run executed **96** rollouts with no abort. The refined actual spend was approximately $3.7, plus approximately $0.07 for the Phase-A calibration, for a **$3.8** total. Each arm carries `N = 20`; at that sample size a single-arm rate has roughly a ±22 pp 95% Wald interval, a small-sample scope we return to in Section 9. The result rests on the by-construction guarantee (N-independent), the 2 empirical gate denials, the 0.20 false-block cost, and the 0.95 conclusion-integrity level — not on any attack-rate delta, of which there is none.

7 Positioning vs SOTA

The harness's contribution is best read along an axis the action-authorization benchmarks do not score: **decision integrity** — does

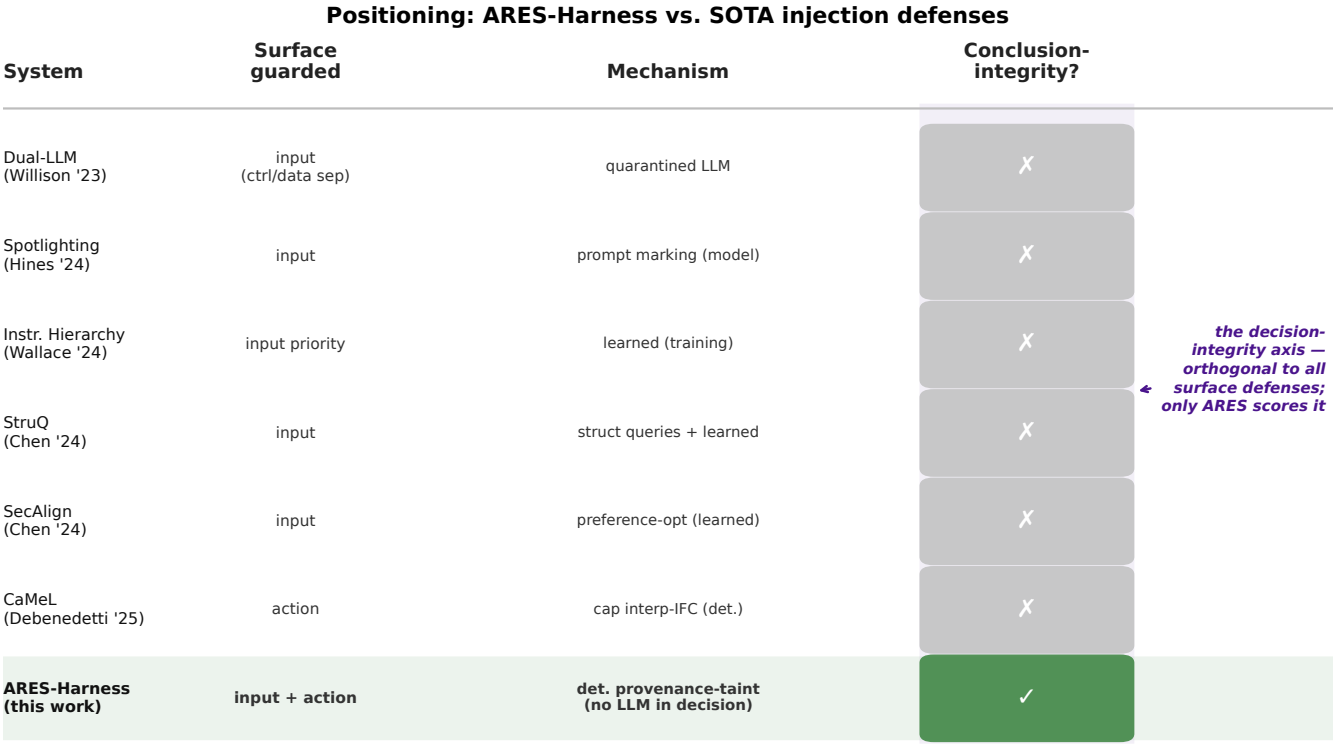

Figure 6: Positioning against SOTA injection defenses by the surface each guards and its mechanism. Model-surface defenses raise a model's learned probability of resisting; action-surface defenses move the security decision out of the model. Only ARES-Harness scores the conclusion-integrity axis, which is orthogonal to the surface each system guards.

Table 4: Positioning against SOTA injection defenses by the surface each guards and its mechanism. Only ARES-Harness scores the conclusion-integrity axis, which is orthogonal to the surface a defense guards.

System	Surface guarded	Mechanism	Concl. integ.?
Dual-LLM (Willison 2023)	input (control/data sep.)	quarantined second LLM	No
Spotlighting (Hines 2024)	input	prompt marking (learned)	No
Instruction hierarchy (Wallace 2024)	input priority	learned prioritization	No
StruQ (Chen 2024)	input	structured queries + learned	No
SecAlign (Chen 2024)	input	preference optimization (learned)	No
CaMeL (Debenedetti 2025)	action	capability interpreter-IFC (deterministic)	No
ARES-Harness (this work)	input + action	deterministic provenance-taint (no LLM in decision)	Yes

the agent's returned conclusion stay honest under injection — alongside the *measured cost* of a deterministic guarantee. Figure 6 and Table 4 place the harness against the field; the positioning is by *surface guarded* and by what each system adds.

7.1 Against CaMeL

CaMeL is the closest relative, and we position against it head-on: both remove the model from the security decision and authorize privileged calls by data provenance rather than model judgment [Debenedetti et al. 2025]. CaMeL compiles the agent's plan to code and enforces information-flow control through a full custom interpreter, tracking provenance through arbitrary intermediate computation, which is a strong and general guarantee. Our gate is deliberately lighter: a single-hop provenance-taint check on each proposed call, with provenance derived harness-side from raw bytes. The trade is explicit and we do not paper over it. CaMeL's interpreter-IFC tracks flows our single-hop check does not — a value laundered through several computational steps before it reaches the call argument — so on adversaries that launder through computation, CaMeL covers cases our gate does not. In exchange, our gate buys two things CaMeL's heavier mechanism does not: it composes around an *unmodified* agent with no compiled execution model, so the integration cost is a wrapper rather than a re-architecting; and

on the scoped environment-state task class it carries the same *kind* of by-construction, non-probabilistic guarantee. The positioning is not "better than CaMeL" — it is "the minimal deterministic action-gate that still yields a by-construction guarantee on the scoped class, at wrapper-level integration cost."

7.2 Against inert-rendering and learned-priority defenses

Against the dual-LLM pattern [Willison 2023] and spotlighting [Hines et al. 2024], our inert rendering shares the control-data separation goal but realizes it differently. The dual-LLM pattern enforces separation by routing untrusted content through a second, quarantined model that the privileged planner never lets touch its execution path; spotlighting marks untrusted spans with a transformation a trained model learns to discount. Our quarantine stage is deterministic and single-process: untrusted content is rendered inert as quoted data by provenance, with no second model and no reliance on the model having learned to discount a marker. The simplification is intentional — there is one fewer model to run and no training dependency — at the cost of inert rendering being a probabilistic mitigation rather than the guarantee, which is why the gate, not the rendering, carries the floor. Against the learned-priority defenses — the instruction hierarchy [Wallace et al. 2024], StruQ [Chen et al. 2024], and SecAlign [Chen et al. 2025] — the contrast is sharper still: those defenses raise a model's *probability* of resisting injection through training, whereas the gate is deterministic code whose denial cannot be argued open by any framing, because the model's text is never consulted in the decision. The two families are complementary rather than competing: a model with a strong learned injection prior, fronting a deterministic gate, fails in independent ways — the prior must be defeated *and* the gate's provenance check evaded — which is the layered posture we recommend.

7.3 The substrate is not novel; its limits are honest

We are explicit that the harness's deterministic ingress scan and named-IOC anchor are not a novel detection substrate — they are regex- and signature-class checks whose evasion behavior is well understood. Their honest limit is exactly the one characterized in the read-depth robustness trilemma: a deterministic signature reader gains detection only by reading more of the attacker-controllable value, and reading more re-acquires the evasion surface, so semantic framing with no signature is a known miss [Gmys-Casiano 2026d]. The harness does not claim to detect every injection. Its claim is *isolation by provenance plus a by-construction action gate*, with the scan as a syntactic pre-filter whose limits we account for rather than oversell. The decision-integrity axis and the deterministic guarantee, not the scan, are the contribution.

8 Discussion

8.1 What a no-headroom regime means for evaluation

The central empirical finding is that on a modern frontier model the strongest action-axis attacks on the banking suite are already saturated-resisted: undefended attack-success-rate is 0.0 across the grid. If that holds broadly — and we are careful to say *if*, given the single-suite scope — the field's dominant evaluation habit of scoring an injection defense by how far it drives attack-success-rate down is measuring in a regime with no room left to move. A real defense and a no-op would score identically on the headline metric, not because the defense does nothing, but because the metric has no headroom for either to move. This is a measurement pathology, not a defense pathology: the number is uninformative precisely where the underlying threat is most resisted. It is why we treat our own degenerate ASR panel as a pre-registered **contingency** rather than a null result. The protocol anticipated the possibility before the run and pre-committed to reading the regime in that case, so a zero everywhere is a *measured finding about the regime* the field operates in, not a failure to find an effect we were hoping for. The methodological consequence is concrete: an injection-defense study on modern agents should pre-register a contingency for the no-headroom case, report the undefended rate prominently so the reader can see whether headroom existed at all, and carry axes that still have signal when the action axis saturates. A paper that only reports a defended-undefended delta in this regime is reporting noise dressed as a result.

8.2 Decision integrity and measured cost as the forward axes

If the action axis is saturated, the axes that still discriminate are *conclusion-integrity* — whether the returned answer stays honest under injection — and the *measured cost* of whatever guarantee a defense provides. We measured both: a 0.95 conclusion-integrity level and a 0.20 pre-registered false-block cost with a `0.50` to `0.30` utility drop. We argue these are the axes the field should evaluate on, and the argument is not merely that they have signal where the action axis does not. Reporting the cost honestly is itself a contribution. A deterministic guarantee that holds an attack at zero by construction is only useful if its price is bounded and known; a defense that quietly over-blocks half of benign traffic is not deployable however strong its guarantee, and the only way to know which side of deployable a defense lands on is to measure and pre-register its cost band. Likewise, conclusion-integrity catches a harm — operator-misleading output — that the action oracle is structurally blind to, and that becomes the *primary* residual harm once the action is blocked. The forward axis we advocate, then, is two-pronged: "what does the guarantee cost, and does the conclusion stay honest," in place of the now-uninformative "by how much did the attack rate drop." A defense paper that answers the first pair tells a deployer something actionable; one that answers only the second, in this regime, tells them nothing.

8.3 Deterministic guarantee versus learned resistance, and when by-construction wins

The deterministic-defense program across the prior ARES papers has repeatedly found the same shape: removing the model from a security decision converts a semantic attack into a data-integrity decision and buys a guarantee the learned approaches cannot match in kind [Gmys-Casiano 2026c,d]. The distinction is qualitative, not

merely a matter of degree. A learned resistance is a raised probability — it can be high, but it is a number that a sufficiently novel input can move, and there is no input-independent floor. A by-construction guarantee is a property of the rule applied to the data, so it has a floor that no input can move within its scope. When the threat is a privileged action whose argument can be traced to untrusted bytes — the environment-state class studied here — the by-construction route wins decisively, because it cannot be eroded by a novel framing and it holds independent of model and of sample size; this is the case in which "by construction" beats any measured delta, because the delta would be measuring a probability where the construction supplies a certainty. When the threat is semantic framing with no traceable signature, the deterministic route reaches its honest limit (Section 7.3), and there isolation, not detection, is what it can offer; a learned resister may then do better on average even without a guarantee. The design lesson is therefore not "deterministic everywhere" but "put the by-construction guarantee where provenance is tractable, lean on isolation where signature detection runs out, and be candid about which regime each part of the system is in." The harness embodies exactly that division of labor.

9 Limitations

We restate the honesty boundary as the first and most important limitation, because the framing of the paper turns on it. The result is a pre-registered **contingency**: because undefended attack-success-rate is 0.0 on this model row and suite, there is no action-ASR headroom, so the harness's benefit is shown **by construction** and by the 2 empirical gate denials, *not* as a measured defended-versus-undefended ASR delta. We do not claim such a delta anywhere in the paper, and a reader should not infer one from the side-by-side arm tables; the arms are equal on ASR by the regime, and the gate's value is the construction plus its denials, not a gap between arms.

Small sample. Each arm carries `N = 20`; a single-arm rate has roughly a ±22 pp 95% Wald interval, and a delta of two rates would carry roughly a ±31 pp interval. Every Stage-1 rate should be read at that resolution, and the 2 gate denials in particular are a small absolute count read as corroboration of the construction, not as an extrapolable rate. **Single setting.** The deciding measurement is one benchmark (AgentDojo), one suite (banking), and one fallback cell (one model, one attack family at the cell selected by the contingency rule); cross-suite, cross-benchmark, and cross-attack-family generalization of both the regime finding and the cost is untested, and a different suite could well show attack headroom that this one does not. **Integrity is a level, not a delta.** Conclusion-integrity is 0.95 across all arms including undefended, so the harness's by-construction inert rendering protects the axis but cannot be credited with a *measured* integrity gain at this sample size on this corpus; the 0.95 is a property of the model-plus-corpus as much as of the defense. **The scan is a syntactic pre-filter.** The deterministic ingress scan does not catch semantic framing that carries no detectable signature, and the named-IOC anchor only resists evasion where the indicator is a named entity; the read-depth robustness trilemma is the honest accounting of that limit, and the harness leans on isolation-by-provenance and the action gate rather than on detection precisely because of it [Gmys-Casiano 2026d]. There is also a residual the provenance derivation does not cover: true semantic laundering, where the attacker's value never appears verbatim in the captured bytes (Section 3.3), lies outside the single-hop value-tracking. **A deferred component ablation.** The finer scan/quarantine/normalize component ablation was deferred — the shipped ingress element has no per-stage knobs and a new-files-only discipline was honored — so `gate_off` versus `full_defense` attributes the gate and the bulk of the utility cost, but not the contribution of the individual ingress stages, which remain measured only in aggregate. None of these bounds undercuts the core claim, which is scoped accordingly: a by-construction guarantee on the environment-state class, an empirically non-vacuous gate, and an honestly measured cost, all reported within a regime we name rather than around it. We have tried throughout to make the paper's reach exactly its grasp.

10 Future Work

Five directions follow from the limitations. First, the **deferred component ablation**: instrument the ingress path with per-stage knobs so scan, quarantine, and normalize can be attributed individually, rather than collapsed into the single `gate_off`-versus-`full_defense` contrast; this would tell a deployer which ingress stages earn their cost. Second, an **out-of-vocabulary adversary fuzz** at the production firewall and IOC rung, using the LLM-proposes / code-disposes adversary methodology from the read-depth program to stress the syntactic pre-filter directly and bound its evasion surface empirically rather than by appeal to the trilemma. Third, **recovering attack-rate headroom**: re-run against a weaker model or a deliberately harder corpus where the undefended action attack is *not* already saturated, so that a defended-versus-undefended delta becomes measurable and the by-construction guarantee can be cross-validated against a non-degenerate baseline — the one experiment that would convert the contingency into a delta. Fourth, a **second suite** beyond banking — the slack or travel suites — to test whether the no-headroom regime and the false-block cost generalize across task domains, and to probe whether a suite with output-echo oracles surfaces the conclusion-integrity axis as a delta. Fifth, an **escalate-to-human hook**: convert a gate denial on a high-value benign task into a confirmation request rather than a silent block, trading some of the 0.20 false-block cost for recovered utility while preserving the guarantee, since a human-confirmed action is no longer authorized solely on tainted provenance. Closing the semantic-laundering residual (Section 3.3) — perhaps with a bounded multi-hop provenance check that tracks a value through a small, fixed number of intermediate computations before it reaches a privileged argument — is a sixth, longer-horizon direction; it would move the gate part way toward CaMeL's interpreter-IFC while keeping the wrapper-level integration cost. Taken together, the first three directions sharpen the present result (attributing cost, bounding the scan's evasion surface, and recovering a measurable baseline), while the last three extend it (cross-suite generalization, a utility-recovering human hook, and a wider provenance guarantee). The natural next session is the contingency-recovery run of the third direction, because it is the single experiment that would let the by-construction guarantee be reported alongside a non-degenerate

measured delta, completing the dual-spine story this paper had to tell from one side only.

References

Samuel Albanie, João F. Henriques, Luca Bertinetto, Alex Hernández-García, Hazel Doughty, and Gül Varol. 2022. Proceedings of the NeurIPS 2021 Workshop on Pre-registration in Machine Learning *(Proceedings of Machine Learning Research, Vol. 181)*. PMLR.

Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2024. StruQ: Defending Against Prompt Injection with Structured Queries. arXiv:2402.06363 [cs.CR] doi:10.48550/arXiv.2402.06363

Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. 2025. SecAlign: Defending Against Prompt Injection with Preference Optimization. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*. arXiv:2410.05451 doi:10.1145/3719027.3744836

Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813 [cs.CR] doi:10.48550/arXiv.2503.18813

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024) Datasets and Benchmarks Track*. arXiv:2406.13352 doi:10.52202/079017-2636

Daniel Gmys-Casiano. 2026a. Decision Determinism, Explanation Drift: Decoupling Verdict Stability from Explanation Stability in Adversarial Multi-Agent LLM Pipelines. *Preprint* (2026).

Daniel Gmys-Casiano. 2026b. The Deterministic Skeptic: Four Rules Match an LLM Agent in Adversarial Cybersecurity Threat Analysis. *Preprint* (2026).

Daniel Gmys-Casiano. 2026c. The Problem Is Inside the Black Box: Asymmetric Calibration Failure in Multi-Agent LLM Debate. *Preprint* (2026).

Daniel Gmys-Casiano. 2026d. Reading Deeper Re-Acquires the Attack: A Pre-Registered Read-Depth Robustness Trilemma for Deterministic Threat Verifiers. *Preprint* (2026).

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*. ACM. doi:10.1145/3605764.3623985

Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. 2024. Defending Against Indirect Prompt Injection Attacks With Spotlighting. arXiv:2403.14720 [cs.CR] doi:10.48550/arXiv.2403.14720

Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208 [cs.CR]

Simon Willison. 2023. The Dual LLM Pattern for Building AI Assistants That Can Resist Prompt Injection. https://simonwillison.net/2023/Apr/25/dual-llm-pattern/.

Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. 2025. Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25)*. arXiv:2312.14197 doi:10.1145/3690624.3709179

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. arXiv:2403.02691

A Generative AI Use Declaration

Generative AI systems were used both as the object of study and as drafting aids, in each case under deterministic gating and human verification. As the object of study: the measured agent is an LLM tool-using agent, and the AgentDojo attacks are LLM-targeted prompt injections; every reported outcome comes from the closed, pre-registered run artifact the paper is reproducible from. The defense itself contains no LLM — the action gate and the ingress path are deterministic code, by design. As a drafting aid: the authors used an LLM coding assistant to scaffold the figure renderer and verification gates and to draft prose, with every load-bearing number locked to the source run artifact by an automated number-check and every claim verified by the authors. No citation, number, or result in this paper was generated without a verifiable source artifact.